

Python code started in 2024 and achieved in 2025 (once more after a lot of struggle, like for project 08) and translated into english in 2026.

The available Python script fulfills the objective described in the *Build it* (§ 5) of the documentation. That is : to be invoked from the command line, to take a `.jack` file (or a directory of `.jack` files) (*) as input to produce a `.vm` file (or a series of corresponding `.vm` files). But it does not follow exactly the proposed *Implementation* of the documentation (§ 11.3). For example, it implements a symbol table but it does not follow the suggested API. Indeed, I preferred to adapt what I had already developed for another personal project written in C.

The code has been retested in 2026 against the following program directories :

Average
ComplexArrays
ConvertToBin
Exemple_forum
Pong
Seven
Square

As one can see, there is one more directory than the 6 described in the documentation. The `Exemple_forum` directory contains `Test.jack` whose code was found on the nand2tetris [forum](#) and which I found useful while trying to solve a particular problem.

Each `.vm` file can be (re)created by calling `JackCompiler.bat` on the corresponding program directory or `.jack` file. Here is an example with `ConvertToBin` :

- Recreate `Main.vm` by using the following kind of command :

```
JackCompiler.bat Perso\ConvertToBin > JackCompiler.ConvertToBin.out
```

- Take a look to `JackCompiler.ConvertToBin.out` to check near the end of the file if the compilation has succeeded :

```
JackCompiler.py, Info, 1 fichier(s) XML traites, 0 erreur(s)
```

- By the way, one can notice this kind of information at the end of the output file. It is the result of the comparison between the `.vm` file (re)created and the equivalent `.vm` file supplied as a resource of the nand2tetris project :

```
Calculating checksums of produced VM files in C:\Temp\JackCompiler.bat.Perso.ConvertToBin.lst...
```

```
Calculating checksums of official VM files in C:\Temp\JackCompiler.bat.ConvertToBin.lst...
```

```
The lists are different : verify if needed with VMEmulator.bat !
```

Since the check sums are different, it can be cautious to compare the 2 files. Hopefully, the only noticeable differences will resemble this :

34 not	34 not
35 not	35 if-goto IF_TRUE0
36 if-goto IF_FALSE0	36 goto IF_FALSE0
	37 label IF_TRUE0
37 push argument 0	38 push argument 0
38 push local 0	39 push local 0
39 and	40 and
40 push constant 0	41 push constant 0
41 eq	42 eq
42 not	
43 not	43 not
44 if-goto IF_FALSE1	44 if-goto IF_TRUE1
	45 goto IF_FALSE1
	46 label IF_TRUE1
45 push constant 8000	47 push constant 8000

What's happening here is that the way I've handled control instructions (like `if` tests or `while` loops) does not produce the exact same code that the one provided in the nand2tetris resources. It is not the same for optimization reasons or because of differences in the labels handling. The important thing (supposing the optimization subject is taken apart) is that the `.vm` produced does behave the way it is supposed to. That's why the final comment suggests to check the final result using the VM Emulator.

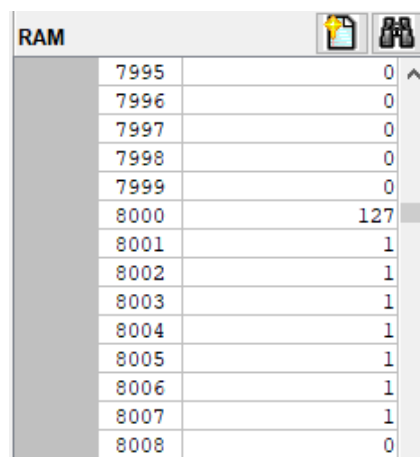
- Once the `Main.vm` file has been recreated, open the VM Emulator (`tools\VMEmulator.bat` on Windows).

- *File / Load Program / Open* `Perso\ConvertToBin\Main.vm` (answering Yes to the confirmation message).

- Locate RAM 8000 address using the binocular search tool and set its value to 127 by double-clicking exactly on the actual value (that is : 0).

- Select *Animate/No animation* to get a fast answer.

- Run the program (`>>`), acknowledge the end of the execution (OK) and press the stop button (■) to observe the effect (`RAM[8001 → 8007] = 1`) :



RAM		
7995		0
7996		0
7997		0
7998		0
7999		0
8000		127
8001		1
8002		1
8003		1
8004		1
8005		1
8006		1
8007		1
8008		0

Another way to (re)create the `.vm` files is to call `JackCompiler_check.bat`. This script can be used as a non-regression test tool as it recompiles the 7 programs mentioned previously and it checks if the `.vm` generated files are identical to previous versions supposed to work properly (those previous versions are suffixed with `.check`).

If all the `.check` files are present, here is the expected execution output :

```
C:\(...)\nand2tetris\projects\11>JackCompiler_check.bat
```

```
Checking Average...
```

```
Comparison between vm files and vm.check files OK !
```

```
-----
```

```
Checking ComplexArrays...
```

```
Comparison between vm files and vm.check files OK !
```

Checking Seven...

Comparison between vm files and vm.check files OK !

Checking ConvertToBin...

Comparison between vm files and vm.check files OK !

Checking Square...

Comparison between vm files and vm.check files OK !

Checking Pong...

Comparison between vm files and vm.check files OK !

Checking Exemple_forum...

Comparison between vm files and vm.check files OK !

If a `.check` file is missing, the script displays the last line of the compilation output. The ideal result expected then is « `OK : the lists are identical !` ». The other possible result is « `The lists are different : verify if needed with VMEmulator.bat !` » (see the previous comments about the `ConvertToBin` program). To display the last line, the script relies on the `tail.exe` program whose directory must be in the current `PATH` and which is part of a series of very useful tools (for those having a minimal knowledge of the Unix commands). Those tools can be freely downloaded [here](#).

(*) Strictly speaking, the `JackCompiler.py` script is *not* expecting a `.jack` file (or a directory of `.jack` files) as input but a `.xml` file (or a directory of `.xml` files). Each `.xml` file being the result of the « tokenization » of a `.jack` file (see project 10). If a `.xml` file is missing, the compilation script calls `JackTokenizer.bat` (which, in turn, calls `JackTokenizer.py`) to create it.